ORGNZ-99: Best Practice Summary & DQL Reference

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/14/2026

Overview

This notebook consolidates every actionable best practice from the ORGNZ series (notebooks 01-10) into a single definitive reference. Each section provides the exact setting, value, or action to take — and a **Validation Queries** subsection with executable DQL to verify your configuration against a live Grail environment.

Table of Contents

1. Bucket Design
2. Retention Strategy
3. Data Routing
4. IAM and Access Control
5. Security Context
6. Segment Design
7. Metadata Enrichment
8. Query Performance
9. Governance and Operations

> Each section ends with a **Validation Queries** block — executable DQL to verify your configuration against a live Grail environment.

---

1. Bucket Design

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Size log buckets for 1–3 TB/day	Logs bucket best practice (08/24/2026): *"between 1 TB and 3 TB of log data per day"*. The older partition-data page (06/17/2026) says to split above 2 TB/day, the only figure stated for other tables	Critical	Performance
2	One data type per bucket	Each bucket stores exactly one table type (logs, metrics, spans, events, bizevents, security.events)	Critical	Architecture
3	Use consistent naming convention	Format: `<provider>_<type>_<retention>` or `<team>_<type>_<retention>` (e.g., `aws_logs_35d`, `team_platform_logs`)	Critical	Naming
4	Bucket names: lowercase, letter-first, underscores/hyphens only	Regex: `^[a-z][a-z0-9_-]*$`; never use `default_*` prefix	Critical	Naming
5	Plan bucket names before creation	Bucket names are immutable after creation; display names can be changed	Critical	Planning
6	Stay within the custom-bucket environment default — 250 from SaaS 1.346 (80 on earlier versions)	Budget buckets across teams; request increase only if justified	Recommended	Planning
7	Attribute cost with cost-allocation fields, not buckets	Set `dt.cost.costcenter` / `dt.cost.product` at the source (works for logs, traces, metrics and events); create a dedicated bucket only where a cost center also needs its own retention or access	Recommended	Cost
8	Create separate buckets for compliance data	Dedicated bucket (e.g., `compliance_audit_logs`) with extended retention for regulatory requirements	Recommended	Compliance
9	Create short-retention buckets for debug logs	Bucket: `debug_logs_7d` with 7-day retention for high-volume verbose logs	Recommended	Cost

#	Best Practice	Recommended Setting/Value	Priority	Category
10	Split high-volume buckets to maintain queryability	If a bucket exceeds the range in row 1, split by line of business first, then by application where volume or compliance justify it	Recommended	Performance
10a	Give an application its own bucket above 250 GB/day	*"Create a dedicated bucket for any application ingesting more than 250 GB/day."* Below that, applications can share a bucket	Recommended	Performance

> **Sources:** Best practices for a logs bucket strategy (DT docs) — row 1 as quoted; *"Split log buckets by line of business rather than by individual application."*, Use Grail buckets to partition data (DT docs) — *"We recommend splitting up buckets that receive more than 2 TB of daily ingest."*, Best practices for data partitioning (DT docs) — row 10a as quoted, Set up Cost Allocation (DT docs) — row 7. Where the pages differ, the newer and more specific page is followed (read 10/14/2026).

Validation Queries — Bucket Design

List all buckets with data type, retention, and estimated storage:

```
// Inventory of all Grail buckets — verify data types, retention, and naming convention
fetch dt.system.buckets
| fields name, display_name, dt.system.table, retention_days, estimated_uncompressed_bytes
| sort dt.system.table asc, name asc
```

List all built-in buckets (use to compare your environment to the expected defaults):

```
// Discover all built-in Grail buckets — compare against expected defaults
fetch dt.system.buckets
| filter startsWith(name, "default_") or startsWith(name, "dt_")
| fields name, display_name, dt.system.table, retention_days
| sort dt.system.table asc, name asc
```

2. Retention Strategy

#	Best Practice	Recommended Setting/Value	Priority	Category
11	Set debug/development log retention to 3-7 days	Bucket retention: 7 days maximum for DEBUG-level logs	Critical	Cost
12	Set standard operational log retention to 35 days	Bucket retention: 35 days for application and infrastructure logs	Recommended	Operations

#	Best Practice	Recommended Setting/Value	Priority	Category
13	Set performance analysis retention to 60-90 days	Bucket retention: 90 days for logs/events used in quarterly trend analysis (metrics retention is fixed at ~15 months in `default_metrics`)	Recommended	Operations
14	Set audit/compliance log retention to 365-3657 days	Bucket retention: match regulatory requirement (SOX: 7 years = 2,555 days; HIPAA: 6 years = 2,190 days)	Critical	Compliance
15	Set span retention to 10-35 days	Bucket retention: 10 days for default APM; 35 days if long-running incident correlation needed	Recommended	Operations
16	Set security event retention to 90-365 days	Bucket retention: 365 days for security investigation timelines	Recommended	Security
17	Never lower retention without understanding data loss	Reducing retention immediately deletes data older than the new setting; this is irreversible	Critical	Operations

Validation Queries — Retention Strategy

Compare retention settings across all bucket types:

```
// Compare bucket retention configurations — verify each tier matches its compliance requirement
fetch dt.system.buckets
| fields name, dt.system.table, retention_days
| sort retention_days desc
```

3. Data Routing

#	Best Practice	Recommended Setting/Value	Priority	Category
18	Route data to custom buckets via OpenPipeline	Configure Storage-stage **Bucket assignment** processors (first match wins) with DQL matchers on `loglevel`, `dt.host_group.id`, `log.source`, `dt.cost.*`	Critical	Architecture
19	Decide where unmatched records go	There is no `default:` route setting; data with no bucket assignment goes to the scope's default bucket (`default_logs` for logs) — check unmatched records in the pipeline preview	Critical	Reliability

#	Best Practice	Recommended Setting/Value	Priority	Category
20	Route DEBUG logs to short-retention buckets	Bucket assignment matcher `loglevel == "DEBUG"` → bucket `debug_logs_7d`	Recommended	Cost
21	Route audit logs to compliance buckets	Bucket assignment matcher `matchesPhrase(log.source, "audit")` → bucket `audit_logs_365d`	Recommended	Compliance
22	Data cannot be moved between buckets after ingestion	Route new data via OpenPipeline; old data stays in the original bucket until retention expires	Critical	Architecture

Validation Queries — Data Routing

Verify routing rules are directing data to the expected buckets:

```
// Audit log volume by bucket — verify routing rules are directing data correctly
fetch logs, from:-1h
| summarize recordCount = count(), by:{dt.system.bucket}
| sort recordCount desc
```

```
// Track log ingest trends by bucket over the last day — identify routing anomalies
fetch logs, from:-24h
| summarize recordCount = count(), by:{time_bucket = bin(timestamp, 1h), dt.system.bucket}
| sort time_bucket desc
```

4. IAM and Access Control

#	Best Practice	Recommended Setting/Value	Priority	Category
23	Always include `storage:buckets:read` in every data access policy	Without bucket read, users cannot query any data; always pair with table permissions	Critical	Security
24	Assign policies to groups, never to individual users	Create IAM groups per team/role; assign all policies at the group level	Critical	Governance
25	Use the two-step policy pattern	Step 1: `ALLOW storage:buckets:read WHERE ...`; Step 2: `ALLOW storage:logs:read` (or other table types)	Critical	Security

#	Best Practice	Recommended Setting/Value	Priority	Category
26	Use STARTSWITH for team bucket prefixes	Policy: `ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "team_platform_"` — automatically includes future buckets with matching prefix	Recommended	Scalability
27	Use MATCH operator for array fields	When `dt.security_context` holds an array, use `MATCH` (not `=`, `STARTSWITH`, or `IN` — those always return false for arrays)	Critical	Security
28	Start with least privilege	Grant minimum required access; expand only when justified	Critical	Security
29	Use policy boundaries for reusable conditions	Define regional/environmental restrictions once as a boundary; apply to multiple policies	Recommended	Governance
30	Limit policy boundaries to 10 restrictions per boundary	Maximum 10 conditions per boundary; create additional boundaries if more are needed	Recommended	Architecture
31	Use fieldsets for sensitive fields	Grant `storage:fieldsets:read` on specific fieldsets (e.g. `builtin-sensitive-spans`, or a custom fieldset for log fields) only where needed — users without it don't see those fields. There is no per-field `DENY` permission	Recommended	Security
32	Test policies with sample users before deployment	Create a test user in the target group and verify query results before rolling out to the team	Critical	Operations
33	Document every IAM policy	Include name, description, tags, target group, and purpose in a policy registry	Recommended	Governance

Validation Queries — IAM and Access Control

Verify bucket access and data distribution across teams:

```
// Verify bucket access — distribution of accessible log data by bucket
fetch logs, from:-1h
| summarize count = count(), by:{dt.system.bucket}
| sort count desc
```

```
// Check namespace-based access — verify namespace-scoped policies return expected data
fetch logs, from:-1h
| filter isNotNull(k8s.namespace.name)
| summarize count = count(), by:{k8s.namespace.name}
| sort count desc
```

```
// Check host group-based access — verify host-group-scoped policies return expected data
fetch logs, from:-1h
| filter isNotNull(dt.host_group.id)
| summarize count = count(), by:{dt.host_group.id}
| sort count desc
```

5. Security Context

#	Best Practice	Recommended Setting/Value	Priority	Category
34	Use `dt.security_context` for record-level access control	Set at the source first (host tag, `DT_TAGS`, K8s namespace labels, OTel resource attribute); OpenPipeline Set security context processor for computed values	Critical	Security
35	Use hierarchical encoding in security context values	Format: `<org>/<department>/<team>` (e.g., `acme/engineering/platform`); enables department-level access with `MATCH ('acme/engineering/*')`	Critical	Scalability
36	Use type prefixes for security context values	Prefix with `team:`, `lob:`, `env:`, `cloud:` (e.g., `team:checkout`, `lob:finance`, `env:production`)	Recommended	Naming
37	Plan security context hierarchy before implementation	Context values are assigned to records at ingest time; changing schema requires re-ingesting data	Critical	Planning
38	Use security context instead of additional buckets when at the bucket-count limit	Security context enables multi-team isolation within shared buckets without consuming bucket quota	Recommended	Scalability
39	Set security context on entities via Grail Security Context settings	Settings → Topology model → Grail Security Context; this scopes the entities only, not their logs/spans/metrics — use a host tag or primary-field enrichment for those	Recommended	Security

#	Best Practice	Recommended Setting/Value	Priority	Category
40	Always use MATCH for array-valued security context in IAM policies	`ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ("team-a*")`	Critical	Security

Validation Queries — Security Context

Verify `dt.security_context` assignment, distribution, and hierarchical patterns:

```
// View logs with security context — confirm dt.security_context is being set
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| fields timestamp, content, dt.security_context
| limit 20
```

```
// Summarize log volume by security context — verify naming conventions and distribution
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize recordCount = count(), by:{dt.security_context}
| sort recordCount desc
| limit 20
```

```
// Shape check — is dt.security_context a single string on every record, or an array on
some?
// Any "array" row means IAM conditions need MATCH and DQL checks need expand (ORGNZ-06).
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| fieldsAdd shape = type(dt.security_context)
| summarize records = count(), by:{shape}
```

```
// Why the checks below expand first — no data scanned. On the array record, == is false
and
// startsWith returns null; after expand, both shapes match.
data record(dt.security_context = array("finance/payments", "team:platform")),
record(dt.security_context = "team:platform"), record(dt.security_context =
"finance/ledger")
| fieldsAdd shape = type(dt.security_context), eq_direct = dt.security_context ==
"team:platform", sw_direct = startsWith(dt.security_context, "finance/")
| expand dt.security_context
| fieldsAdd eq_expanded = dt.security_context == "team:platform", sw_expanded =
startsWith(dt.security_context, "finance/")
```

```
// Count logs carrying one security-context value. expand first: dt.security_context can be an array,
// and == against an array is always false, so a bare filter would miss exactly those records.
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| expand dt.security_context
| filter dt.security_context == "team:platform"
| summarize records = count()
```

```
// Which records a MATCH ("finance/*") policy would cover — this does not test the policy itself.
// expand handles string and array values alike; startsWith on an array returns null.
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| expand dt.security_context
| filter startsWith(dt.security_context, "finance/")
| summarize count = count(), by:{dt.security_context}
| sort count desc
```

```
// Audit security context coverage — percentage of logs with dt.security_context assigned
fetch logs, from:-1h
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    withoutContext = countIf(isNull(dt.security_context))
| fieldsAdd coverage = round(toDouble(withContext) / toDouble(total) * 100, decimals: 2)
```

6. Segment Design

#	Best Practice	Recommended Setting/Value	Priority	Category
41	Use segments for dynamic data views, not access control	Segments filter at query time; they do not enforce security boundaries — IAM policies do	Critical	Architecture
42	Stay within the 20-include limit per segment	Maximum 20 include blocks per segment; plan rule complexity accordingly	Critical	Architecture
43	One include block per data type	Cannot have two `logs` includes; combine conditions with `OR` within a single include	Critical	Architecture

#	Best Practice	Recommended Setting/Value	Priority	Category
44	Use Primary Grail Fields in segment filters	Filter on `dt.host_group.id`, `k8s.namespace.name`, `k8s.cluster.name`, `aws.account.id`, `azure.subscription`, `gcp.project.id` — Dynatrace enriches them on every signal from a source that has them; check coverage per data type, because problems can carry far fewer than logs (ORGNZ-10 §3)	Critical	Performance
45	Use prefix-based naming for cross-signal filtering	Name host groups, namespaces, etc. with prefixes (e.g., `prod-web-tier` not `web-tier-prod`) so a prefix (`= "prod-*"`) works on signal includes and on classic `entity.name` (the only classic-entity property that accepts starts-with)	Recommended	Naming
46	Create one segment per dimension	Separate segments for platform, application, environment, and region — users combine them as needed	Recommended	Architecture
47	Add event includes for detected problem filtering	Define events include with `event.kind = "DAVIS_PROBLEM"` — entity includes alone do not filter problems	Critical	Troubleshooting
48	Set official segments to Public visibility	Platform-team segments: *Anyone in the environment*; application-team segments: *Unlisted*, surfaced through the team's shared notebooks and dashboards (segments have no share-with-group setting)	Recommended	Governance
49	Use consistent segment naming prefixes	Prefixes: `team-`, `env-`, `app-`, `region-` (e.g., `team-platform-infra`, `env-production`)	Recommended	Naming
50	Test segment filter conditions with DQL before creating the segment	Run the equivalent `filter` clause as a DQL query; verify record counts match expectations	Critical	Operations
51	Limit segment variable dropdowns to 10,000 values	Maximum 10,000 values per variable; 100 selected values per segment	Recommended	Architecture

#	Best Practice	Recommended Setting/Value	Priority	Category
52	Use entity relationship traversal for comprehensive segments	Top-down: host → process group → service; bottom-up: service → process group → host	Recommended	Architecture
53	Segments reduce scanned bytes	Applying a segment injects filters at query time, reducing data scanned and improving performance on high-volume buckets	Recommended	Performance

Validation Queries — Segment Design

Simulate segment filter conditions before committing to a segment definition:

```
// Simulate host-group-based segment filter on logs
fetch logs, from:-1h
| filter isNotNull(dt.host_group.id)
| summarize count = count(), by:{dt.host_group.id}
| sort count desc
| limit 10
```

```
// Simulate a primary-tag segment filter on the service list: what a Data (all types)
include sees.
// Service metrics carry primary Grail tags, not classic entity tags. Replace
primary_tags.app
// with your key. Validated 10/14/2026 with a tenant-specific key: 3 values, 4-17 services
each.
timeseries requests = sum(dt.service.request.count), from:-2h, by:{primary_tags.app,
dt.smartscape.service}
| filter isNotNull(primary_tags.app)
| fieldsAdd total = arraySum(requests)
| summarize {services = countDistinct(dt.smartscape.service), requests = sum(total)}, by:
{primary_tags.app}
| sort requests desc
```

```
// Simulate combined segment filter — verify host group + namespace combination returns
expected logs
fetch logs, from:-1h
| filter isNotNull(dt.host_group.id) and isNotNull(k8s.namespace.name)
| summarize count = count(), by:{dt.host_group.id, k8s.namespace.name}
| sort count desc
| limit 20
```

7. Metadata Enrichment

#	Best Practice	Recommended Setting/Value	Priority	Category
54	Enrich all signals with Primary Grail Fields before building segments	Verify `dt.host_group.id`, `k8s.namespace.name`, `k8s.cluster.name` are present on logs, spans, and metrics	Critical	Architecture
55	Set `dt.security_context` via a host tag for dedicated infrastructure	Host tag `dt.security_context=<team>` (installation, `oneagentctl --set-host-tag`, or Deployment Status)	Recommended	Security
56	Set `dt.cost.costcenter` and `dt.cost.product` on hosts	Host tags: `dt.cost.costcenter=<center>`, `dt.cost.product=<product>` — these propagate to service metrics	Recommended	Cost
57	Use `DT_TAGS` environment variable for shared infrastructure	Set `DT_TAGS=primary_tags.team=<team>` per process when multiple apps share the same host	Recommended	Architecture
58	Restart applications after enrichment changes for spans	Logs: no restart needed (OneAgent auto-enriches); Spans: application restart required; Metrics: OneAgent restart applies automatically	Critical	Operations
59	Verify enrichment coverage before building segments	Run DQL audit query to check field coverage percentage across signal types; fields with 0% coverage need enrichment	Critical	Operations
60	Use Primary Grail Tags (`primary_tags.*`) when standard fields do not match your organization	Set via **host tags** (`hostautotag.conf` / `oneagentctl --set-host-tag`) or the `DT_TAGS` environment variable; propagates across logs, spans, and topology. **Host properties no longer work for this** — see row 60a	Recommended	Architecture

#	Best Practice	Recommended Setting/Value	Priority	Category
60a	Do not set host-level `primary_tags.*` via `hostcustomproperties.conf`	**Breaking, OneAgent 1.345:** *"the OS Agent derives `primary_tags` from host tags (`hostautotag.conf`) rather than from host properties (`hostcustomproperties.conf`) at the host level."* Existing primary tags configured through host properties silently stop being promoted once a host reaches 1.345 — audit before the fleet upgrades, and migrate them to host tags	Critical	Migration

Validation Queries — Metadata Enrichment

Audit primary Grail field coverage before building segment definitions:

```
// Audit primary Grail field coverage on logs — low-coverage fields need enrichment before segment use
fetch logs, from:-1h
| summarize
    total = count(),
    with_namespace = countIf(isNotNull(k8s.namespace.name)),
    with_cluster = countIf(isNotNull(k8s.cluster.name)),
    with_host_group = countIf(isNotNull(dt.host_group.id)),
    with_security_ctx = countIf(isNotNull(dt.security_context))
| fieldsAdd
    ns_pct = round(toDouble(with_namespace) / toDouble(total) * 100, decimals: 1),
    cluster_pct = round(toDouble(with_cluster) / toDouble(total) * 100, decimals: 1),
    hg_pct = round(toDouble(with_host_group) / toDouble(total) * 100, decimals: 1),
    sec_pct = round(toDouble(with_security_ctx) / toDouble(total) * 100, decimals: 1)
```

```
// Audit primary Grail field coverage on spans — spans require app restart to pick up new enrichment
fetch spans, from:-1h
| summarize
    total = count(),
    with_namespace = countIf(isNotNull(k8s.namespace.name)),
    with_cluster = countIf(isNotNull(k8s.cluster.name)),
    with_host_group = countIf(isNotNull(dt.host_group.id)),
    with_service = countIf(isNotNull(dt.smartscape.service))
| fieldsAdd
    ns_pct = round(toDouble(with_namespace) / toDouble(total) * 100, decimals: 1),
    cluster_pct = round(toDouble(with_cluster) / toDouble(total) * 100, decimals: 1),
    hg_pct = round(toDouble(with_host_group) / toDouble(total) * 100, decimals: 1),
    svc_pct = round(toDouble(with_service) / toDouble(total) * 100, decimals: 1)
```

```
// Variable DQL: list host groups. Use as a segment variable for host-group-scoped
filters.
// There is no HOST_GROUP Smartscape node type; the host group is the dt.host_group.id
field
// on HOST nodes. Validated 10/14/2026; 0 bytes scanned.
smartscapeNodes "HOST"
| filter isNotNull(dt.host_group.id)
| fields host_group = dt.host_group.id
| dedup host_group
| sort host_group asc
```

```
// Variable DQL: list Kubernetes namespaces. Use as a segment variable for namespace-
scoped filters.
// Validated 10/14/2026: 12 namespace names across 23 K8S_NAMESPACE nodes; 0 bytes
scanned.
smartscapeNodes "K8S_NAMESPACE"
| fields namespace = name
| dedup namespace
| sort namespace asc
```

8. Query Performance

#	Best Practice	Recommended Setting/Value	Priority	Category
61	Understand the default 500 GB scan limit	`fetch` stops at 500 GB scanned by default (`scanLimitGBytes`); at 1 TB/day ingest an unmodified query reaches back ~12 hours — raise `scanLimitGBytes` or narrow the query	Critical	Performance
62	Target specific buckets in queries	Use `fetch logs, bucket:{"team_logs", "app_logs_*"}` to avoid scanning all buckets	Recommended	Performance
63	Use exact match (`=`) over pattern match in segment filters	`=` is faster than `starts-with` ; use exact values when known	Recommended	Performance
64	Minimize segment filter expressions	Fewer conditions = better query performance; maximum 10 expressions per filter	Recommended	Performance
65	Use OpenPipeline pre-processing for complex segment logic	If segment conditions would be complex, pre-enrich a simple field (e.g., `dt.security_context`) and filter on that	Recommended	Performance

#	Best Practice	Recommended Setting/Value	Priority	Category
66	Know the default result limits: 1,000 records and 1 MB	Dynatrace lists *"1,000 records returned per result"* and *"1 MB result payload"* among its default query limits; aggregate (`summarize` , `makeTimeseries`) or raise `maxResultRecords` / `maxResultBytes` on a Query API request	Recommended	Performance
67	Maximum 10 segments per query	Do not stack more than 10 segments; design broader segments instead of many narrow ones	Recommended	Architecture

> **Sources:** Best practices for a logs bucket strategy (DT docs) — row 66 as quoted, Grail storage query SDK (Dynatrace Developer) — `maxResultRecords` and `maxResultBytes` request parameters.

Validation Queries — Query Performance

Use bucket targeting to control scan scope and stay within the 500 GB scan limit:

```
// Query from multiple buckets simultaneously — reduce scan scope by naming targets explicitly
// Bucket names that don't exist are skipped with no warning — a misspelled name looks like an empty bucket.
// Check names first: fetch dt.system.buckets | filter dt.system.table == "logs" | fields name
fetch logs, from:-1h, bucket:{"default_logs", "audit_logs", "security_logs"}
| limit 100
```

```
// Filter by bucket within query — useful when bucket list is dynamic
fetch logs, from:-1h
| filter dt.system.bucket == "default_logs"
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel == "ERROR" misses them
| filter status == "ERROR"
| limit 50
```

9. Governance and Operations

#	Best Practice	Recommended Setting/Value	Priority	Category
68	Schedule regular access reviews	Quarterly review of all IAM policies and group memberships; remove unnecessary permissions	Critical	Governance

#	Best Practice	Recommended Setting/Value	Priority	Category
69	Document all bucket purposes	Maintain a registry mapping each bucket to its owner, data type, retention, cost center, and routing rule	Recommended	Governance
70	Document all security context values	Maintain a registry of all `dt.security_context` values, their meaning, and which IAM policies reference them	Recommended	Governance
71	Platform team owns infrastructure segments	Platform/SRE team creates and maintains public environment and infrastructure segments	Recommended	Governance
72	Application teams own app-specific segments	Each team creates segments for its applications, kept *Unlisted* and referenced from the team's shared documents	Recommended	Governance
73	Review segments quarterly	Remove unused segments; verify filter conditions still match data; check for empty results	Recommended	Operations
74	Use combined approach for defense in depth	Buckets for retention/cost + Security context for access control + Segments for dynamic views — all three together	Critical	Architecture
75	Implement in phases	Phase 1: Plan → Phase 2: Buckets + routing → Phase 3: IAM policies → Phase 4: Segments → Phase 5: Governance	Recommended	Operations
76	Selective record deletion is not possible	Cannot delete individual records; wait for bucket retention to expire, or truncate the entire bucket	Critical	Operations
77	Bucket deletion destroys all data permanently	Deleting a bucket removes ALL records immediately and irreversibly	Critical	Operations

Validation Queries — Governance and Operations

Routine audit queries for operational health checks:

```
// Audit bucket data distribution — verify all expected buckets are receiving data
fetch logs, from:-1h
| summarize
    recordCount = count(),
    by:{dt.system.bucket}
| sort recordCount desc
```

```
// Audit bucket retention settings — verify retention tiers match compliance requirements
fetch dt.system.buckets
| fields name, dt.system.table, retention_days
| sort dt.system.table asc, retention_days desc
```

Audit bucket management events — who created, updated, truncated, or deleted a bucket and when:

```
// Audit all bucket management actions — create, update, truncate, delete
// Bucket changes are rare: widen to from:-365d for a full year of history
fetch dt.system.events, from:-90d
| filter event.kind == "AUDIT_EVENT" and event.category == "BUCKET_MANAGEMENT"
| fields timestamp, event.type, resource, user.id, event.outcome
| sort timestamp desc
```

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.